JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class LocaleServiceProvider

java.lang.Object
    java.util.spi.LocaleServiceProvider

Direct Known Subclasses:

BreakIteratorProvider, CalendarDataProvider, CalendarNameProvider, CollatorProvider, CurrencyNameProvider, DateFormatProvider, DateFormatSymbolsProvider, DateTimeFormatterPatternProvider, DecimalFormatSymbolsProvider, LocaleNameProvider, NumberFormatProvider, TimeZoneNameProvider

---

```
public abstract class LocaleServiceProvider
extends Object
```

This is the super class of all the locale sensitive service provider interfaces (SPIs).

Locale sensitive service provider interfaces are interfaces that correspond to locale sensitive classes in the `java.text` and `java.util` packages in order to provide the locale data used for each service. The interfaces enable the construction of locale sensitive objects and the retrieval of localized names for these packages. Locale sensitive factory methods and methods for name retrieval in the `java.text` and `java.util` packages use implementations of the provider interfaces to offer support for locales beyond the set of locales supported by the Java runtime environment itself. Locale sensitive service providers are deployed on the application module path or the application class path. In order to be looked up, providers must be visible to the `system class loader`. See `Deploying Service Providers` for further detail on deploying a locale sensitive service provider as a module or on the class path.

Packaging of Locale Sensitive Service Provider Implementations

For a locale sensitive service provider deployed in a module, the *provides* directive must be specified in the module declaration. The *provides* directive specifies both the service and the service provider.

For example, an implementation of the `DateFormatProvider` class deployed as a module might specify the following directive:

```
    provides java.text.spi.DateFormatProvider with com.example.ExternalDateFormat
```

For a Locale Service Provider deployed on the class path, the provider identifies itself with a provider-configuration file in the resource directory META-INF/services. The file name should be the fully fully qualified provider

interface class name. The file should contain a list of fully-qualified concrete provider class names, one per line. A line is terminated by any one of a line feed ('\n'), a carriage return ('\r'), or a carriage return followed immediately by a line feed. Space and tab characters surrounding each name, as well as blank lines, are ignored. The comment character is '#' ('#'); on each line all characters following the first comment character are ignored. The file must be encoded in UTF-8.

If a particular concrete provider class is named in more than one configuration file, or is named in the same configuration file more than once, then the duplicates will be ignored. The configuration file naming a particular provider need not be in the same jar file or other distribution unit as the provider itself. The provider must be accessible from the same class loader that was initially queried to locate the configuration file; this is not necessarily the class loader that loaded the file.

For example, an implementation of the `DateFormatProvider` class should take the form of a jar file which contains the file:

```
META-INF/services/java.text.spi.DateFormatProvider
```

And the file `java.text.spi.DateFormatProvider` should have a line such as:

```
com.foo.DateFormatProviderImpl
```

which is the fully qualified class name of the class implementing `DateFormatProvider`.

Invocation of Locale Sensitive Services

Locale sensitive factory methods and methods for name retrieval in the `java.text` and `java.util` packages invoke service provider methods when needed to support the requested locale. The methods first check whether the Java runtime environment itself supports the requested locale, and use its support if available. Otherwise, they call the `isSupportedLocale` methods of installed providers for the appropriate interface to find one that supports the requested locale. If such a provider is found, its other methods are called to obtain the requested object or name. When checking whether a locale is supported, the locale's extensions are ignored by default. (If a locale's extensions should also be checked, the `isSupportedLocale` method must be overridden). If neither the Java runtime environment itself nor an installed provider supports the requested locale, the methods go through a list of candidate locales and repeat the availability check for each until a match is found. The algorithm used for creating a list of candidate locales is same as the one used by `ResourceBundle` by default (see `getCandidateLocales` for the

details). Even if a locale is resolved from the candidate list, methods that return requested objects or names are invoked with the original requested locale including `Locale` extensions. The Java runtime environment must support the root locale for all locale sensitive services in order to guarantee that this process terminates.

Providers of names (but not providers of other objects) are allowed to return null for some name requests even for locales that they claim to support by including them in their return value for `getAvailableLocales`. Similarly, the Java runtime environment itself may not have all names for all locales that it supports. This is because the sets of objects for which names are requested can be large and vary over time, so that it's not always feasible to cover them completely. If the Java runtime environment or a provider returns null instead of a name, the lookup will proceed as described above as if the locale was not supported.

The search order of locale sensitive services can be configured by using the `java.locale.providers` system property. This system property declares the user's preferred order for looking up the locale sensitive services separated by a comma. As this property value is read and cached only at the initialization of this class, users should specify the property on the java launcher command line. Setting it at runtime with `System.setProperty(String, String)` is discouraged and it may not affect the order. JDK Reference Implementation provides the following three locale data providers:

- "CLDR": A locale data provider based on the Unicode Consortium's Common Locale Data Repository (CLDR).
- "SPI": represents the locale sensitive services implementing the subclasses of this `LocaleServiceProvider` class.
- "HOST": A locale data provider that reflects the user's custom settings in the underlying operating system. This provider may not be available, depending on the JDK Reference Implementation.

For example, if the following is specified in the property:

```
java.locale.providers=SPI,CLDR
```

the locale sensitive services in the SPI providers are looked up first. If the desired locale sensitive service is not available, then the runtime looks for CLDR.

The default value for looking up the preferred locale data providers is "CLDR", so specifying only "CLDR" is identical to the default behavior. Applications which require implementations of the locale sensitive services must explicitly specify "SPI" in order for the Java runtime to load them from the classpath.

Implementation Note:

The JDK uses locale data from the Unicode Consortium's Common Locale Data Repository (CLDR) to implement locale-sensitive APIs in the `java.util` and `java.text` packages. This locale data derives the set of locales supported by the Java runtime environment. The following tables list the version of CLDR used in each JDK release. Unless otherwise specified, all update releases in a given JDK release family use the same CLDR version. Note that the CLDR locale data are subject to change. Users should not assume that the locale data remain the same across CLDR versions. Otherwise, unexpected incompatible behaviors may occur, such as an exception on parsing a date. Refer to CLDR Releases for the deltas between their releases.

JDK release	CLDR version
JDK 27	CLDR 48.2
JDK 25	CLDR 47
JDK 21	CLDR 43
JDK 17	CLDR 39
JDK 11	CLDR 33
JDK 8	CLDR 21.0.1

Show other JDK releases

JDK release	CLDR version
JDK 26	CLDR 48
JDK 24	CLDR 46
JDK 23	CLDR 45
JDK 22	CLDR 44
JDK 20	CLDR 42
JDK 19	CLDR 41
JDK 18	CLDR 39
JDK 16	CLDR 38
JDK 15	CLDR 37
JDK 14	CLDR 36
JDK 13	CLDR 35.1
JDK 12	CLDR 33
JDK 10	CLDR 29
JDK 9	CLDR 29

Since:

1.6

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	`LocaleServiceProvider()`	Initializes a new locale service provider.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
abstract `Locale[]`	`getAvailableLocales()`	Returns an array of all locales for which this locale service provider can provide localized objects or names.
boolean	`isSupportedLocale(Locale locale)`	Returns `true` if the given `locale` is supported by this locale service provider.

Methods declared in class Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `toString`, `wait`, `wait`, `wait`

Constructor Details

LocaleServiceProvider

```
protected LocaleServiceProvider()
```

Initializes a new locale service provider.

Method Details

getAvailableLocales

```
public abstract Locale[] getAvailableLocales()
```

Returns an array of all locales for which this locale service provider can provide localized objects or names. This information is used to compose `getAvailableLocales()` values of the locale-dependent services, such as `DateFormat.getAvailableLocales()`.

The array returned by this method should not include two or more `Locale` objects only differing in their extensions.

Returns:

an array of all locales for which this locale service provider can provide localized objects or names

isSupportedLocale

`public boolean isSupportedLocale(Locale locale)`

Returns `true` if the given `locale` is supported by this locale service provider. The given `locale` may contain extensions that should be taken into account for the support determination.

The default implementation returns `true` if the given `locale` is equal to any of the available `Locales` returned by `getAvailableLocales()` with ignoring any extensions in both the given `locale` and the available locales. Concrete locale service provider implementations should override this method if those implementations are `Locale` extensions-aware. For example, `DecimalFormatSymbolsProvider` implementations will need to check extensions in the given `locale` to see if any numbering system is specified and can be supported. However, `CollatorProvider` implementations may not be affected by any particular numbering systems, and in that case, extensions for numbering systems should be ignored.

Parameters:

`locale` - a `Locale` to be tested

Returns:

`true` if the given `locale` is supported by this provider; `false` otherwise.

Throws:

`NullPointerException` - if the given `locale` is `null`

Since:

1.8

See Also:

`Locale.hasExtensions()`, `Locale.stripExtensions()`

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which

contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.

Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.

Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.

All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.